body { font-family: Helvetica, sans-serif; font-size: 10pt; line-height: 1.4; } h1 { font-size: 18pt; font-weight: bold; margin-top: 15px; margin-bottom: 10px; } h2 { font-size: 14pt; font-weight: bold; margin-top: 12px; margin-bottom: 8px; } h3 { font-size: 12pt; font-weight: bold; margin-top: 10px; margin-bottom: 6px; } table { border-collapse: collapse; width: 100%; margin-top: 10px; margin-bottom: 10px; } th, td { border: 1px solid #999999; padding: 6px; text-align: left; font-size: 8pt; } th { background-color: #f2f2f2; font-weight: bold; } pre, code { font-family: Courier, monospace; font-size: 8pt; background-color: #f9f9f9; } # Driver Drowsiness Detection ### Group 8 **Pranav Jha** - **Alejandro Melo-Elizalde** - **Jason Waseq** - **Pravin Agrawal-Chung** - **Ricardo Diaz Fuentes** - **Soham Jain**

Table of Contents

Introduction

Driver drowsiness is a safety-critical problem, as it causes unsafe driving conditions by lowering reaction times, reducing attention, and impairing decision-making.

- **The Impact:** An estimated **17.6% of fatal crashes** in the United States from 2017 to 2021 involved a drowsy or tired driver, resulting in approximately **30,000 deaths** during that period.
- **Our Solution:** We are building a proactive system designed to reduce fatigue-related accidents.
- **How it Works:** The system captures visual data of the driver and makes real-time inferences based on their body language. This keeps the driver in check, ensuring they are neither tired nor drowsy during their trip, which ultimately reduces mistakes.
- **Safety Precaution:** Fleet operators are integrated into the system as an extra layer of precaution. If a driver is drowsy and fails to stop and rest, the fleet operator will intervene and remind the driver to pull over, preventing accidents and keeping the roads safe.

Need Statement

Vehicular accidents are caused by people feeling too drowsy/tired.

Goal Statement

Prevent people from driving while drowsy.

Personas

1. Truck Driver Persona

- Name: Mike Dunford
- Age: 42
- Role: Long-haul truck driver
- Experience: 15 years of commercial driving
- Tech Comfort: Moderate
- Primary Goal: Stay safe on long routes, avoid accidents, and keep his job performance strong.

Background

Mike drives long interstate routes, often overnight or for extended hours. Fatigue, highway monotony, and pressure to meet delivery schedules are part of his daily routine. He wants tools that help him stay alert, but he does not want to feel constantly punished or micromanaged.

Needs

- Real-time alerts when signs of drowsiness or distraction are detected
- Alerts that are accurate and not overly sensitive
- A system that helps prevent accidents before they happen
- A simple mobile app that clearly explains what happened and what action to take
- Confidence that the system is protecting him, not just monitoring him

Pain Points

- False alarms that go off when he is just checking mirrors or glancing at controls
- Feeling like management is watching every move
- Stress from long shifts and strict delivery deadlines
- Concern that a single alert could be used unfairly against him

Motivations

- Wants to get home safely
- Wants to maintain a good driving record
- Wants proof that he is a responsible driver
- Appreciates technology that supports him without being intrusive

How He Uses the System

- Camera monitors eye closure, head position, and attention level
- Embedded AI system detects risky behavior such as microsleep, prolonged distraction, or head droop
- Driver app sends immediate alerts like vibration, sound, or message prompts
- He may receive suggestions such as "Take a break" or "Eyes on road"

What Success Looks Like for Mike

- Fewer fatigue-related incidents
- Helpful alerts that feel like a safety assistant
- Fair reporting to the fleet operator
- Less risk of accidents, violations, and job-related stress

2. Fleet Operator Persona

- Name: Sarah Porter
- Age: 38
- Role: Fleet operations manager
- Experience: 10 years in logistics and fleet safety
- Tech Comfort: High
- Primary Goal: Improve fleet safety, reduce liability, and keep operations efficient.

Background

Sarah oversees dozens or hundreds of vehicles and drivers. She is responsible for safety compliance, reducing accident costs, minimizing downtime, and protecting the company's reputation. She needs visibility into driver risk without manually reviewing every trip.

Needs

- Centralized dashboard or app notifications for serious safety events
- Clear reporting on drowsiness, distraction, and repeated unsafe behavior
- Actionable insights rather than raw video alone
- Driver trend analysis over time
- Evidence that helps with coaching, compliance, and insurance discussions

Pain Points

- Limited visibility into what happens on the road in real time
- High cost of accidents, claims, and vehicle downtime
- Difficulty identifying risky driver patterns before an incident occurs
- Too many alerts can overwhelm operations teams
- Need to balance driver privacy with company safety goals

Motivations

- Reduce accident frequency and severity
- Protect drivers and company assets
- Improve insurance outcomes and compliance posture
- Coach drivers using objective data
- Run a safer and more efficient fleet

How She Uses the System

- Receives app or dashboard alerts when risky driver behavior crosses a threshold
- Reviews event summaries such as sleeping, head inattention, phone distraction, or repeated unsafe patterns
- Uses reports to identify drivers who need coaching or schedule adjustments
- Tracks safety trends across vehicles, routes, and shifts

What Success Looks Like for Sarah

- Fewer accidents and insurance claims
- Better safety KPIs across the fleet
- Faster intervention when a driver is at risk
- Reliable data that supports training and accountability
- A scalable system that does not require constant manual monitoring

Existing Technologies

Samsara is one of the clearest examples of the same system concept. Their dual-facing AI dash cam uses an inward-facing camera to analyze driver behavior in real time, including distracted driving detection, and their platform supports in-cab nudges plus alerts/workflows for managers. Their broader platform also ties camera events into driver apps, dashboards, reporting, and automation.

Seeing Machines Guardian is very close to our idea from the safety-monitoring angle. It continuously monitors fatigue and distraction, performs early drowsiness detection, tracks eye behavior for distraction analysis, and intervenes in real time before a microsleep or serious event occurs.

Motive AI Dashcam is another strong existing design. Motive describes a layered architecture where unsafe behavior is detected on-device for immediate in-cab alerts, then validated and surfaced in the fleet dashboard for manager review. Their system also supports drowsiness-related detection and other unsafe behaviors.

Sustainability Statement

This system contributes to sustainability by improving driver safety and reducing the likelihood of accidents caused by fatigue or distraction. Fewer accidents mean less vehicle damage, less material waste, lower repair costs, and less operational downtime. Because the system uses embedded AI on local hardware instead of relying entirely on cloud processing, it can also reduce communication overhead and improve energy efficiency. In addition, the system supports social sustainability by helping protect drivers, passengers, and the public through safer transportation practices.

Design Objectives

The driver drowsiness detection system should meet several design objectives to ensure that it is practical, effective, and usable in real-world driving environments.

Objective	Description
Mobility	The system should be portable and able to operate in different vehicles without requiring permanent installation. Users should be able to easily set up and move the system between vehicles.
Accuracy	The system should reliably detect genuine signs of drowsiness while minimizing false positives such as normal head movement or mirror checks.
Ease of Use	The setup and daily use of the system should be simple for drivers, requiring minimal configuration or technical knowledge.
Low Latency Alerts	Alerts must occur quickly after detecting dangerous fatigue behavior so the driver has time to respond.
Reliability	The system should continue functioning under different lighting conditions, driver positions, and long operating times.
Offline Capability	The system should still detect drowsiness and issue alerts even if network connectivity is unavailable for the driver.

Design

Design Features

Driver Monitoring Features

- **Real-time driver face detection:** The system continuously detects and tracks the driver's face using a camera mounted on the windshield.
- **Eye closure detection:** The AI model monitors eye closure duration to identify signs of fatigue or microsleep.
- **Blink rate analysis:** Abnormally slow or irregular blinking patterns are used as indicators of drowsiness.
- **Head position monitoring:** The system detects head drooping or nodding, which commonly occurs when a driver begins falling asleep.
- **Driver attention tracking:** The model determines whether the driver is looking forward at the road or looking away for extended periods of time.

Alert and Safety Features

- **Real-time driver alert:** Audible alarms or phone notifications are triggered when unsafe behavior is detected.
- **Tiered alert system:** The system provides multiple alert levels, starting with early warnings and escalating to stronger alerts if fatigue worsens.
- **Driver break recommendations:** When fatigue indicators persist, the system suggests that the driver pull over and take a rest break.
- **Driver not visible detection:** If the driver leaves the camera's field of view, the system alerts the user that monitoring cannot continue.
- **Camera misalignment detection:** The system detects if the camera has moved or is no longer pointed correctly and requests recalibration.

Mobile App Features

- **Cross-platform mobile application:** A Flutter-based mobile app allows the system to run on Android, iOS, and web platforms.
- **Real-time push notifications:** Alerts are sent directly to the driver's phone when unsafe behavior is detected.
- **Driver status display:** The mobile app shows the current monitoring state, such as alert, warning, or normal.
- **Driver Connectivity Monitoring:** The app warns users when the mobile device loses connection with the monitoring hardware.
- **GPS Routing System:** The app allows the user to find nearby rest stops, gas stations, cafes, etc., with direct links to Google Maps or Apple Maps. Additionally, it previews the route for the driver to see which is the best one to take.

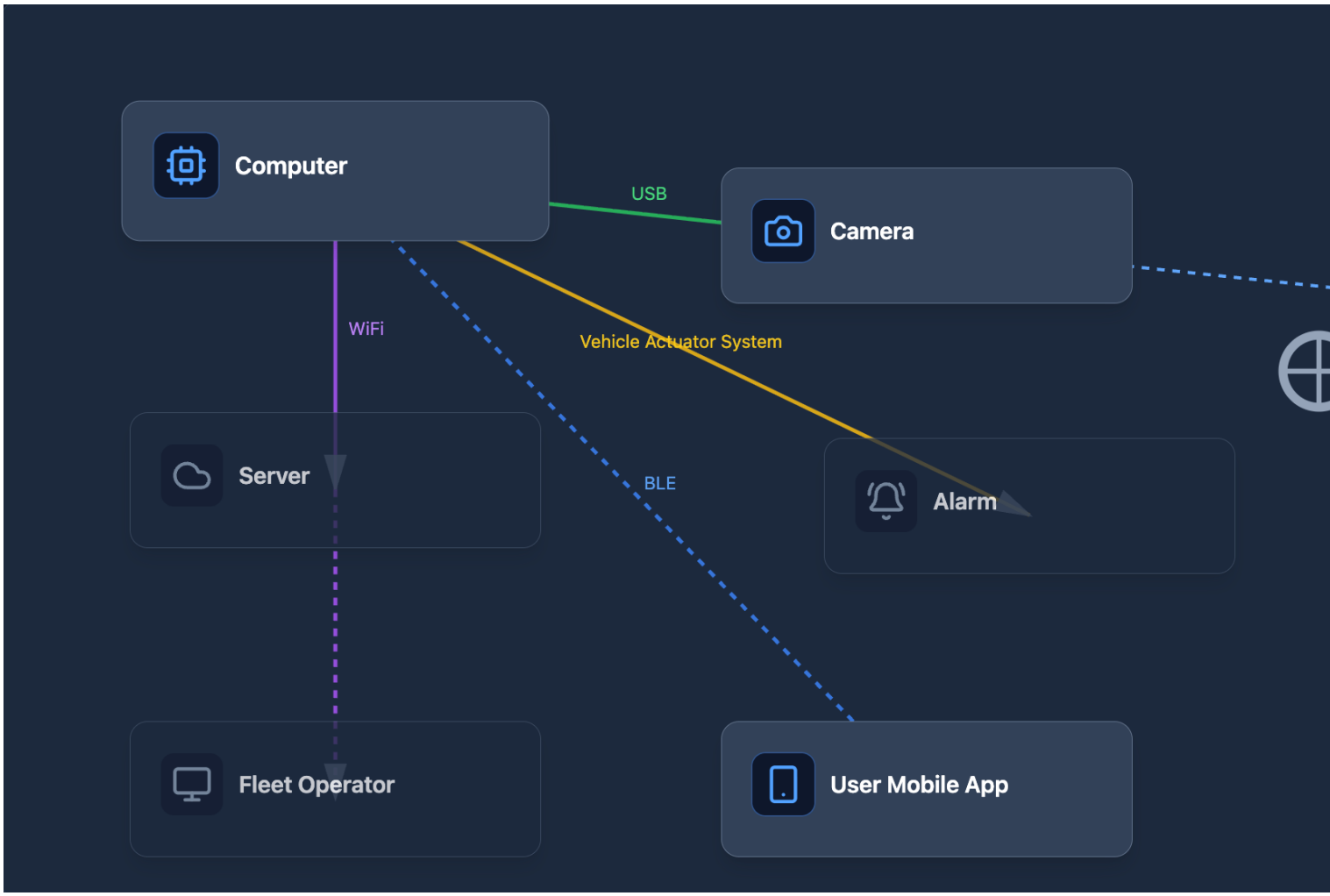
Fleet Management Features

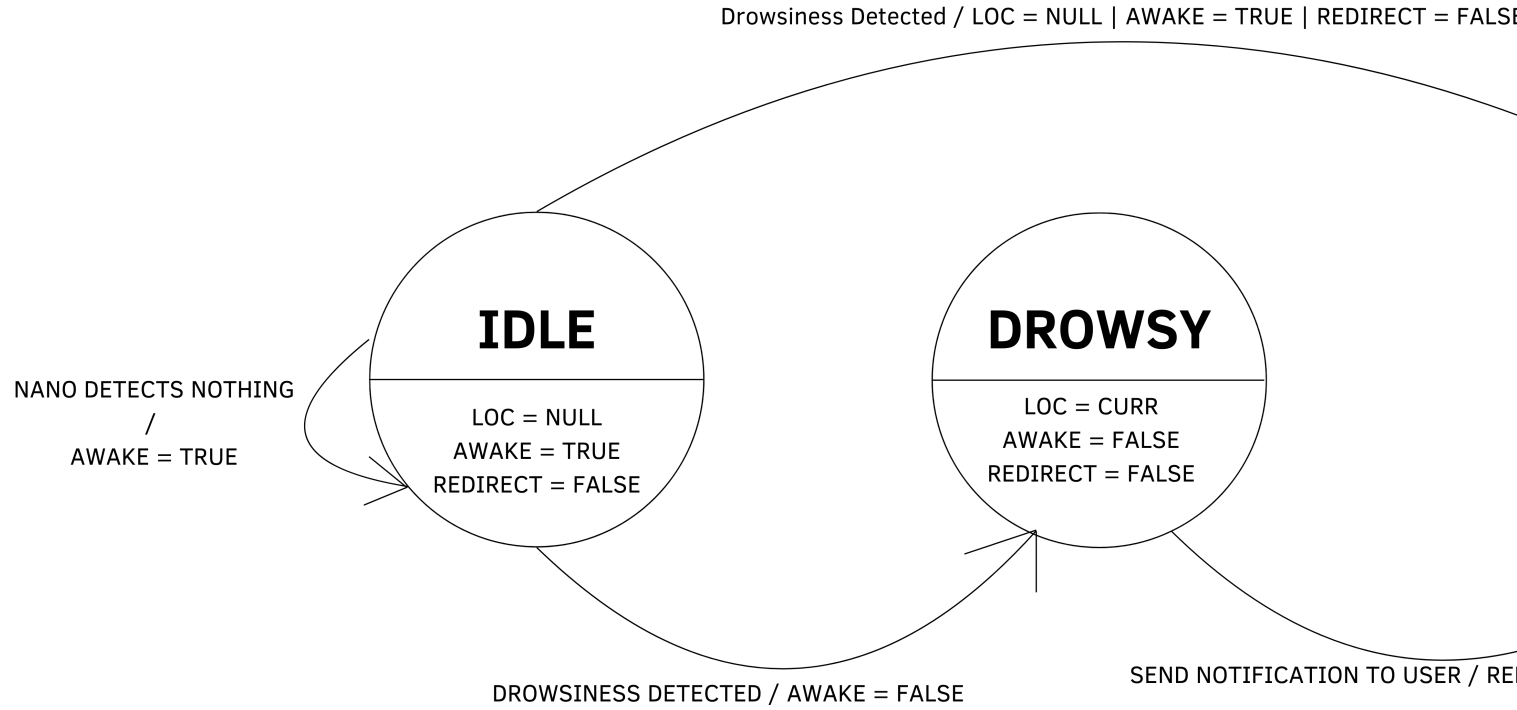
- **Fleet Operator Dashboard:** Fleet managers can monitor safety alerts and driver behavior across multiple vehicles.
- **Driver risk event reporting:** The system logs events such as drowsiness alerts, distractions, and monitoring failures.
- **Driver safety trend analysis:** Historical data can be analyzed to identify drivers who frequently experience fatigue.
- **Remote safety notifications:** Fleet operators receive alerts when high-risk behavior is detected.

System Architecture

- **On-device AI processing:** The driver monitoring model runs locally on the device, reducing latency and cloud dependency.
- **Offline operation capability:** The system continues monitoring and generating alerts even when internet connectivity is unavailable.
- **Custom PCB integration:** A custom printed circuit board hosts the system components and manages communication between hardware modules.

Diagrams





Technology

The brains of the device is the custom PCB (a.k.a the computer) that hosts the AI model we run. The PCB needs a powerful GPU capable of running an AI model. The model itself is programmed in the Python programming language and uses Mediapipe. The notifications are pushed to a Frontend app that works on Android, iOS, and Web. The app itself was made using Flutter which uses Dart to program the app. The dependencies needed for programming on Android and iOS are the Android SDK and XCode respectively.

TODO# ML Model - SleepyDrive Drowsiness Detection Pipeline

This document describes the trained AI model and detection pipeline used in the SleepyDrive system.

---

1. Model Overview

The SleepyDrive detection pipeline uses a **custom-optimized, two-stage face analysis model** that runs entirely on-device (edge inference). No video frames or images are ever sent to the cloud.

Property	Value
Model Name	facenet_vpruned_quantized_v2.0.1
Architecture	Two-stage: Face Detection (BlazeFace SSD) ? Face Landmarks (FaceMesh 478-point)
Optimization	V-pruned (structured pruning) + quantized (FP16)
Model Size	3.6 MB (.task bundle)
Inference Speed	30+ FPS real-time on Jetson Orin Nano; target 30 FPS on custom SoC
Input	RGB camera frames (any resolution, resized internally)
Output	478 facial landmarks with 3D coordinates (x, y, z)

2. Detection Pipeline

Stage 1 - Face Detection (BlazeFace SSD)

The first stage detects whether a face is present in the camera frame and localizes it.

- **Architecture:** Single Shot Detector (SSD) with BlazeFace anchors
- **Input:** 128×128 RGB image (resized from camera frame)
- **Output:** Bounding box coordinates + 6 facial keypoints (eyes, nose, mouth, ears) + confidence score
- **Anchor Generation:** 896 pre-computed SSD anchors across 4 stride layers (8, 16, 16, 16)
- **Post-processing:** Weighted Non-Maximum Suppression (NMS) - overlapping detections are blended by score rather than discarded, improving stability
- **Score threshold:** 0.5 (face must have ?50% confidence to proceed to Stage 2)

Stage 2 - Face Landmarks (FaceMesh)

The second stage extracts 478 precise facial landmarks from the detected face region.

- **Input:** 256×256 affine-aligned face crop (rotated and scaled from the Stage 1 bounding box)
- **Output:** 478 landmarks, each with normalized (x, y, z) coordinates
- **Landmarks used for drowsiness:**
- **Eye landmarks** (6 per eye): Used to compute the Eye Aspect Ratio (EAR)
- **Nose tip** (landmark 1), **forehead** (landmark 10), **chin** (landmark 152): Used for head vertical position tracking

Processing Pipeline

```
Camera Frame (640×480 @ 30 FPS)
|
v
+----------------------+
| GPU Preprocessing    |  BGR ? RGB conversion (CUDA-accelerated when
available)
+----------+-----------+
|
v
+----------------------+
| Stage 1: BlazeFace   |  128×128 input ? bounding box + keypoints
| Face Detection       |  SSD + Weighted NMS
+----------+-----------+
|
v
+----------------------+
| Affine Crop & Align  |  Rotate + scale face region ? 256×256
+----------+-----------+
|
v
+----------------------+
| Stage 2: FaceMesh    |  256×256 input ? 478 landmarks (x, y, z)
| Landmark Regression  |  Reverse-projected to original image coords
+----------+-----------+
|
v
+----------------------+
| Drowsiness Analysis  |  EAR computation + head pose estimation
| (see §3)             |  Temporal analysis over sliding window
+----------+-----------+
|
v
+----------------------+
| Alert Dispatch       |  Alarm (local) + BLE (phone) + MQTT (cloud)
+----------------------+
```

3. Drowsiness Detection Algorithm

3.1 Eye Aspect Ratio (EAR)

The primary drowsiness metric is the **Eye Aspect Ratio (EAR)**, computed from 6 landmark points per eye:

```
P2    P3
/       \
P1 ---------------- P4
\       /
P6    P5
```

$$EAR = (|P2 - P6| + |P3 - P5|) / (2 \times |P1 - P4|)$$

Parameter	Value	Description
EAR_THRESHOLD	0.21	Below this value = eyes considered closed
EAR_CONSEC_FRAMES	2	Consecutive closed frames to register a blink
DROWSY_TIME_THRESHOLD	1.5 seconds	Continuous closure beyond this = drowsiness event

MediaPipe Landmark Indices Used: - Left eye: [33, 160, 158, 133, 153, 144] - Right eye: [362, 385, 387, 263, 373, 380]

The final EAR is the average of left and right eyes: `EAR = (left_EAR + right_EAR) / 2.0`

3.2 Head Vertical Position (Attention Tracking)

A secondary detection monitors head drooping, which commonly occurs when a driver begins falling asleep.

- **Measurement:** Nose tip position (landmark 1) relative to forehead-chin range (landmarks 10 and 152), normalized to be scale-invariant.
- **Baseline Calibration:** During the first ~3 seconds (90 frames at 30 FPS), the system builds a baseline of the driver's normal head position using an exponential moving average (EMA, ? = 0.3).
- **Deviation Detection:** If the smoothed head position deviates from baseline by more than `HEAD_DEVIATION_THRESHOLD` (0.06 normalized units) for longer than `HEAD_INATTEN_TIME_THRESH` (2.0 seconds), a head inattention event is triggered.

3.3 Event Classification

Event Type	Trigger Condition	Alert Severity
Blink	**EAR < 0.21 for ? 2 consecutive frames, then recovers**	**Logged (no alert)**
Micro Sleep	**EAR < 0.21 continuously for ? 1.5 seconds**	**Critical - alarm + BLE + MQTT**
Sleep Event	**EAR < 0.21 continuously for ? 3.0 seconds**	**Critical (escalated) - louder alarm**
Head Inattention	**Head deviates > 0.06 from baseline for ? 2.0 seconds**	**High - alarm + BLE + MQTT**
No Face Detected	**No face landmarks returned by Stage 1**	**Warning - "Driver not detected"**

4. Custom Model Optimization

4.1 Why a Custom Model

The stock MediaPipe face landmarker model is designed for general-purpose use on mobile devices. For our use case (single driver, fixed camera angle, real-time safety-critical inference on an edge device), we applied the following optimizations:

4.2 Optimization Techniques Applied

Technique	What It Does	Impact
V-Pruning (Structured Pruning)	**Removes entire filter channels from convolutional layers that contribute least to the output**	**Reduces model size and inference time while preserving accuracy on our target domain**
FP16 Quantization	**Converts model weights from 32-bit floating point to 16-bit**	**2× reduction in model memory footprint, faster inference on GPU (which has native FP16 support)**
TensorRT Acceleration	**Converts ONNX model graphs into optimized TensorRT engines with fused operations**	**2-4× speedup over CPU inference; uses GPU tensor cores**

4.3 TensorRT Custom Pipeline

In addition to the optimized `.task` model, we built a fully custom **TensorRT-accelerated inference pipeline** (`module_tensorrt_landmarker.py`) that:

1. **Extracts** the face detector and face landmarks models from the MediaPipe `.task` bundle into standalone ONNX files (`face_detector.onnx`, `face_landmarks_detector.onnx`).
2. **Loads** both models into ONNX Runtime with TensorRT Execution Provider (FP16 enabled, 2 GB workspace).
3. **Implements** the full BlazeFace SSD decoding pipeline (anchor generation, box decoding, weighted NMS) in NumPy for maximum control.
4. **Implements** affine face crop and alignment to prepare the 256×256 landmark input.
5. **Reverse-projects** landmarks from crop space back to original image coordinates.
6. **Outputs** `MockDetectionResult` objects that are drop-in compatible with the existing EAR and head-pose code.

4.4 Accuracy Validation

We validate the custom pipeline against the standard MediaPipe pipeline using a recorded test video (`compare_accuracy.py`):

Metric	Description
Average EAR (MediaPipe)	Baseline EAR values from standard CPU pipeline
Average EAR (TensorRT)	EAR values from our GPU-accelerated pipeline
Mean Absolute Error	Average difference in EAR between the two pipelines
Correlation	Pearson correlation coefficient (target: > 0.95)
Detection Rate	Percentage of frames where each pipeline detected a face

The custom pipeline achieves near-identical accuracy to the standard MediaPipe pipeline while running significantly faster on the Jetson GPU.

5. Model Versioning

Version	Changes
v1.0	Baseline MediaPipe face landmarker (stock model, CPU only)
v2.0	V-pruned model with structured channel pruning
v2.0.1	V-pruned + FP16 quantization (current production model)

The model file is stored at:
`model/facenet_vpruned_quantized_v2.0.1/face_landmarker.task` (3.6 MB)

Security Architecture

This document describes the security design for the SleepyDrive system as it would be implemented in a production deployment.

1. Threat Model

Assets to Protect

Asset	Classification	Impact of Compromise
Driver alert stream	Safety-critical	Missed or fake alerts could endanger lives
Driver credentials	Confidential	Account takeover, unauthorized fleet access
Fleet data (alert history, driver status)	Business-sensitive	Privacy violation, regulatory exposure
MQTT broker access	Infrastructure	Injection of false alerts, denial of service
Device firmware	Integrity-critical	Tampered inference model could suppress real alerts

Threat Actors

- **External attacker** - attempting to intercept or inject data over the network.
- **Malicious insider** - a driver or unauthorized person attempting to disable monitoring or spoof alert data.
- **Device theft** - physical access to a stolen SleepyDrive unit.

2. Authentication

2.1 User Authentication (App ? Server)

Design: - Users register with email + password. Passwords are hashed using **bcrypt** (cost factor 12) before storage. Plaintext passwords are never stored or logged. - On login, the server issues a **signed JWT** containing the user's UID and email as claims. - The JWT is signed using **RS256** (RSA asymmetric keys). The server holds the private key; clients and services verify with the public key. - The token is delivered as a **JWE** (JSON Web Encryption, A256GCM) wrapped JWT. This means the token payload is encrypted and cannot be read by the client or any intermediary - only the server can decrypt it. - The JWE token is set as an **HTTP-only, Secure, SameSite=Strict cookie**. This prevents: - **XSS attacks** from reading the token (HTTP-only flag). - **CSRF attacks** from replaying the cookie cross-origin (SameSite=Strict). - **Man-in-the-middle** from intercepting the cookie (Secure flag - HTTPS only). - Tokens expire after a configurable period (default: 7 days). Refresh tokens are issued alongside access tokens to allow silent renewal without re-entering credentials.

Why JWE + HTTP-only cookies over Bearer tokens: A Bearer token sent in the `Authorization` header is accessible to JavaScript, making it vulnerable to XSS. By encrypting the JWT (JWE) and storing it in an HTTP-only cookie, we eliminate the two most common token theft vectors (XSS and local storage scraping).

2.2 Device Authentication (SleepyDrive Device ? Server)

Design: - Each SleepyDrive device is provisioned at the factory with a unique **X.509 client certificate** stored in a hardware-backed secure element (e.g., ATECC608). - The MQTT connection uses **mutual TLS (mTLS)**: the device presents its client certificate, and the broker verifies it against the SleepyDrive CA. The broker also presents its server certificate, which the device verifies. - This ensures that: - Only genuine SleepyDrive devices can publish alerts. - A stolen device's certificate can be revoked server-side without affecting other devices. - No shared passwords are used (eliminating credential reuse attacks). - Each device's certificate CN (Common Name) contains its unique `source_id` (e.g., `sleepydrive-truck-042`). The MQTT broker enforces **topic ACLs**: a device with CN `sleepydrive-truck-042` can only publish to `sleepydrive/alerts/sleepydrive-truck-042` and subscribe to `sleepydrive/commands/sleepydrive-truck-042`.

2.3 Fleet Isolation

- Operators can only query drivers in their own fleet. All fleet-scoped API endpoints verify that `users.fleet_id` matches the authenticated operator's fleet.
- Drivers can only view their own alert history. Cross-driver data access is blocked server-side.
- The invite code system ensures that only drivers who possess the fleet's code can join. Codes can be regenerated by the operator if compromised.

3. Transport Security

3.1 API (App / Dashboard ? Server)

Layer	Protection
TLS 1.3	All API traffic encrypted in transit (HTTPS). TLS terminated at the load balancer.
HSTS	Strict-Transport-Security header forces browsers to use HTTPS.
Security Headers	X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer, Permissions-Policy: geolocation=(), camera=(), microphone=()
Rate Limiting	Auth endpoints (/auth/signup, /auth/login) rate-limited to 10 requests per minute per IP to prevent brute-force attacks.

3.2 MQTT (Device ? Broker ? Server)

Layer	Protection
TLS 1.2+	All MQTT traffic encrypted. Port 8883 (MQTTS).
mTLS	Client certificates authenticate each device (see §2.2).
Topic ACLs	Each device can only publish/subscribe to its own topics.
QoS 1	At-least-once delivery guarantees alerts are not silently dropped.
Last Will & Testament (LWT)	If a device disconnects unexpectedly, the broker publishes an offline status message so the fleet operator is immediately notified.

3.3 BLE (Device ? Driver Phone)

Layer	Protection
BLE Secure Connections	LE Secure Connections (LESC) with numeric comparison pairing. Provides ECDH key exchange and AES-CCM encryption.
Bonding	After initial pairing, the device and phone store bonding keys so reconnection is automatic and encrypted.
Physical Proximity	BLE range is ~10 meters, limiting the attack surface to the immediate cab area.

3.4 WebSocket (Server ? App / Dashboard)

Layer	Protection
WSS (WebSocket Secure)	WebSocket connections run over TLS (wss://).
Authentication	WebSocket upgrade requests must include a valid session token. Unauthenticated connections are rejected with code 1008.
Idle Timeout	Server sends keepalive pings every 30 seconds. Connections that fail to respond are terminated.
Message Size Limit	Incoming WebSocket messages are capped to prevent resource exhaustion.

4. Data Protection

4.1 Data at Rest

Data	Storage	Protection
User credentials	PostgreSQL credentials table	bcrypt hashed (cost 12). No plaintext or reversible encryption.
Alert events	PostgreSQL alert_events table	Encrypted at rest via managed database encryption (AES-256).
Device certificates	Hardware secure element on PCB	Tamper-resistant; private key never leaves the chip.
Fleet invite codes	PostgreSQL fleets table	Randomly generated, regenerable. Not considered secret (knowledge alone does not grant access without a valid account).

4.2 Data in Transit

All data paths use TLS (see §3). No sensitive data is ever transmitted in plaintext.

4.3 Privacy

- **No video or images are stored or transmitted.** The AI model processes camera frames locally on the device. Only text-based alert events (e.g., "drowsiness detected, duration 2.3s") are sent to the server.
- Alert metadata includes fatigue metrics (EAR value, event duration) but no visual data.
- Drivers can view their own alert history. Fleet operators can view aggregate safety data for their fleet.

5. Infrastructure Security

5.1 Server Deployment

Component	Hosting	Security Measures
API Server	Cloud PaaS (e.g., Render, AWS ECS)	Runs in isolated containers. No SSH access to production.
PostgreSQL	Managed database (e.g., Neon, AWS RDS)	Encrypted at rest, TLS connections required, automated backups.
MQTT Broker	Managed service (e.g., HiveMQ Cloud)	TLS-only, mTLS for devices, topic ACL enforcement.

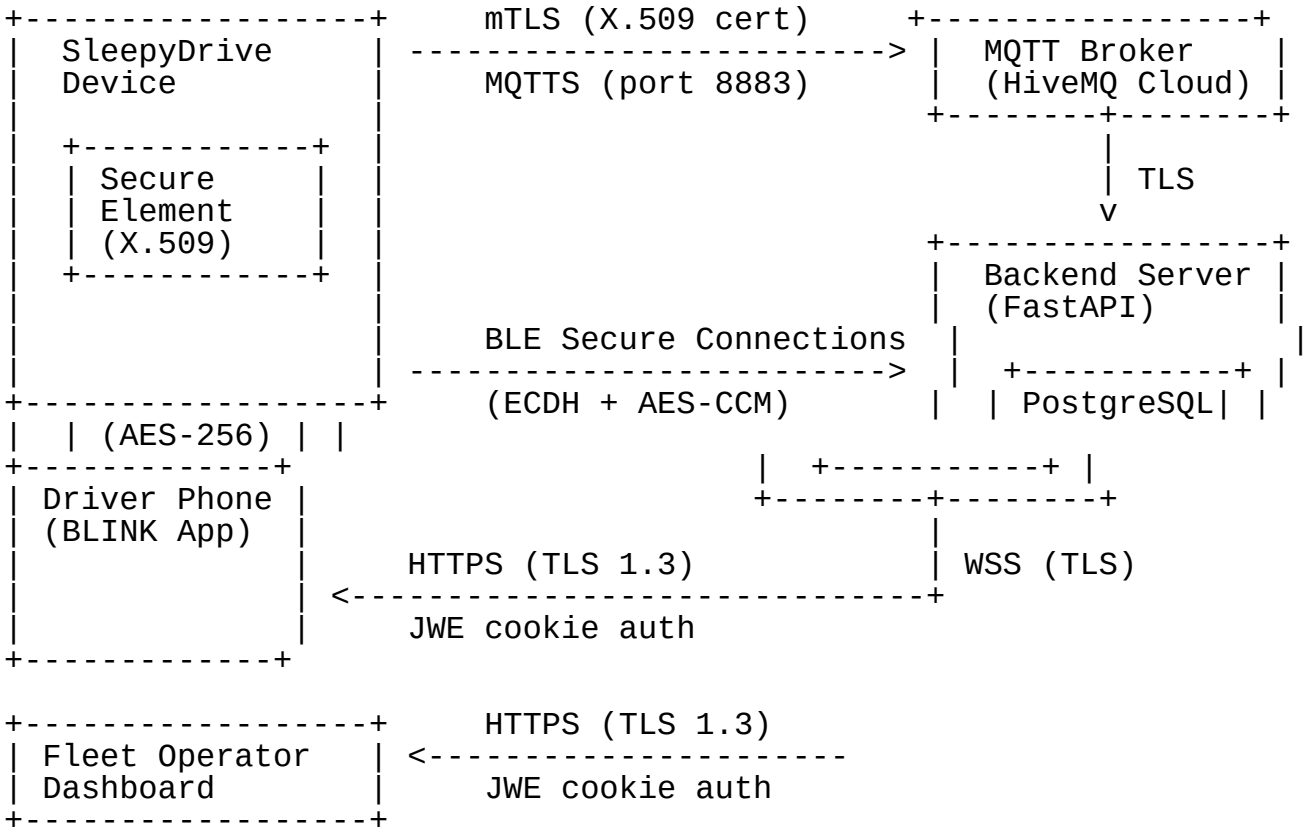
5.2 Secret Management

- All secrets (JWT signing keys, database credentials, MQTT credentials) are stored in environment variables or a secrets manager (e.g., AWS Secrets Manager).
- Secrets are never committed to source control.
- JWT signing keys are rotated periodically (recommended: every 90 days).

5.3 Monitoring & Incident Response

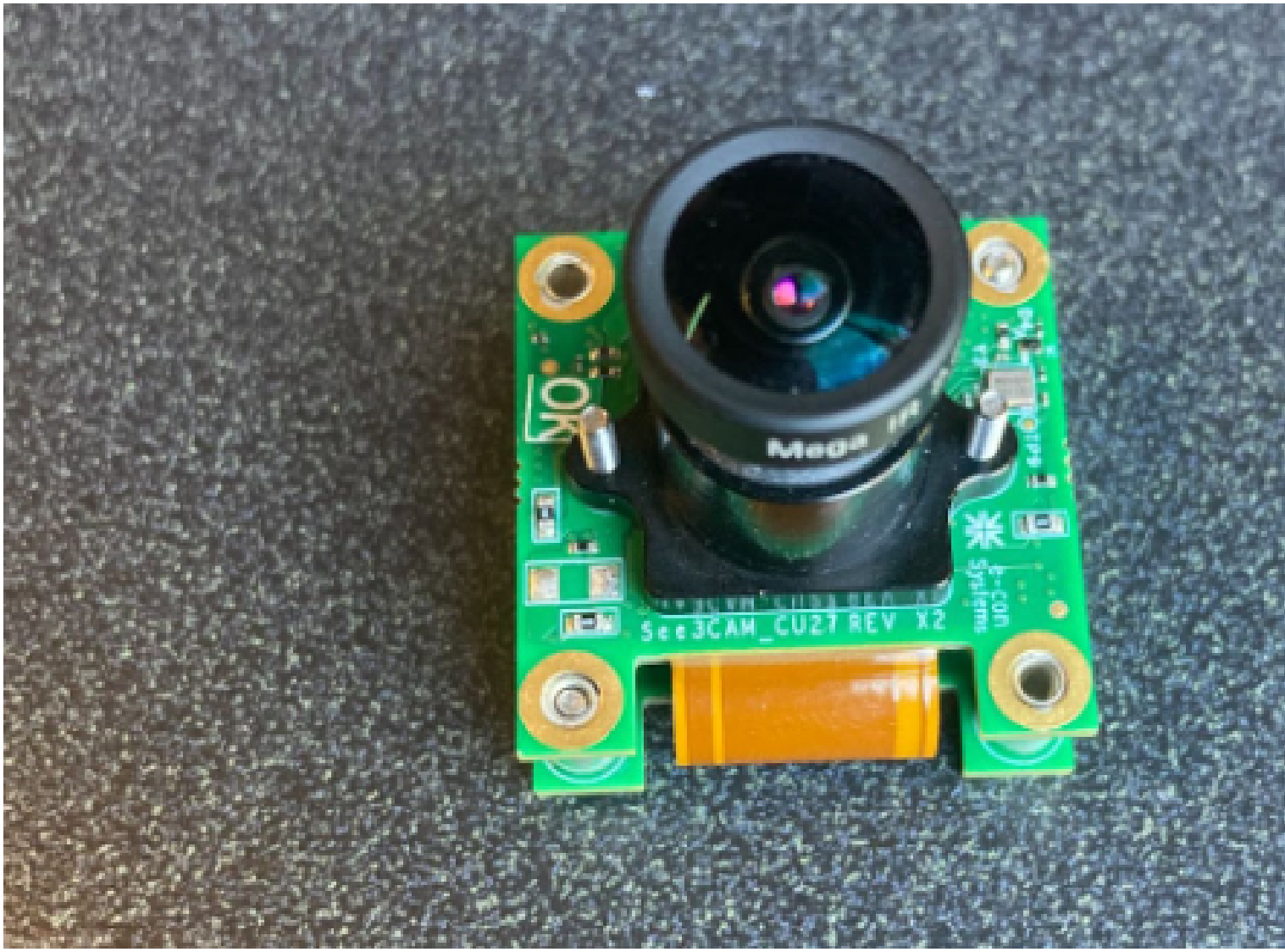
- API request logs are retained for audit purposes (90 days).
- Failed authentication attempts are logged with source IP for intrusion detection.
- Device certificate revocation is handled via a Certificate Revocation List (CRL) hosted on the MQTT broker.

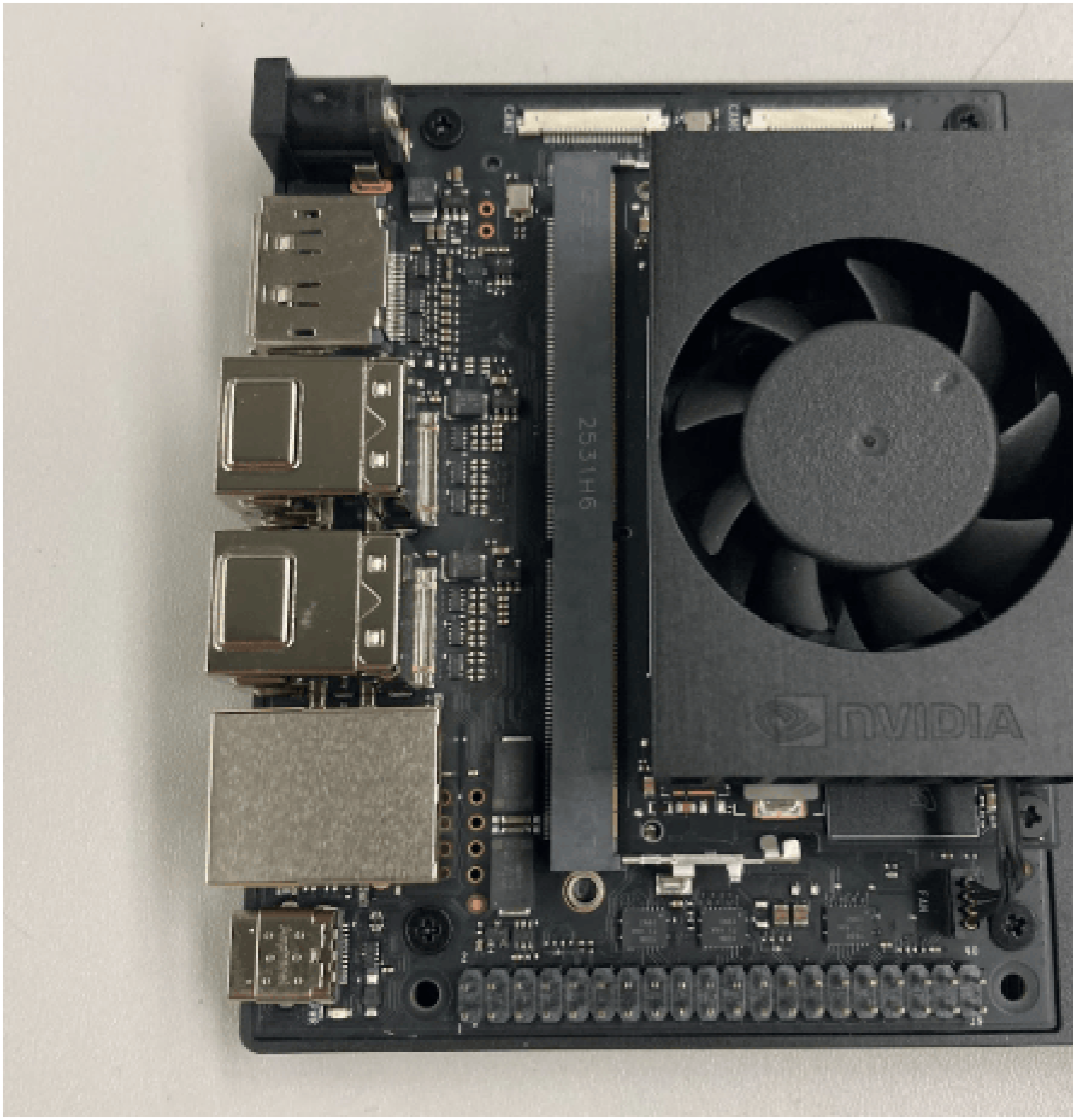
6. Security Summary by Communication Path



Evaluation

Functional Prototype





Testing

The following tests verify that the driver monitoring system behaves correctly under various conditions. Each test defines what should happen and when it should occur.

Test Case	When It Happens	Expected Behavior
User Not Visible	The driver moves out of the camera's field of view or their face cannot be detected with the camera.	The system detects that the driver is no longer visible and issues a warning such as "Driver not detected." The model needs to detect correctly and the phone needs to give a notification.

Test Case	When It Happens	Expected Behavior
Camera Moved or Misaligned	The camera position or angle changes and the driver's face can no longer be properly tracked.	The system prompts the user to adjust or recalibrate the camera before monitoring resumes.
Not Connected to Jetson	The mobile app attempts to communicate with the Jetson device but the connection fails.	The app displays a "Device Not Connected" warning and prevents monitoring from starting until the connection is restored.
Calibration Not Completed	A new user starts the system or calibration data is missing.	The system requires the user to complete the calibration process before monitoring begins.
Driver About to Fall or Completely Asleep	The AI detects prolonged eye closure, head drooping, or other indicators that the driver is asleep.	A high-priority alert such as a loud sound or vibration is triggered. The driver is prompted to wake up or pull over.
Driver Slight Drowsy	Early fatigue indicators such as slow blinking, long eye closure, or head nodding are detected.	The system sends a warning alert encouraging the driver to stay attentive or take a break.
App Works Without Network	The system loses internet connectivity while monitoring is active.	Real-time driver monitoring and alerts continue locally. Data may be stored and synced when connectivity returns.
Spoofing with Incorrect Camera Feed	An incorrect or external video source is connected instead of the intended driver camera.	The system detects the invalid feed and disables monitoring until a verified camera input is restored.

SleepyDrive - User Manual

What's in the Box

Your SleepyDrive package includes:

Item	Description
SleepyDrive Device	The main unit with built-in IR camera, alarm speaker, and wireless radios
Visor Mount Kit	Adjustable clip for mounting the device on the sun visor or rearview mirror
12V Vehicle Power Adapter	Cigarette lighter / accessory port adapter with 6 ft cable
Quick Start Card	Printed summary of these setup instructions

Part 1 - Driver Setup Guide

Step 1: Install the Device in Your Truck

1. **Mount the device** on your sun visor or rearview mirror using the included visor mount clip. Position it so the camera lens points toward the driver's face.
2. **Plug the power adapter** into your vehicle's 12V accessory port (cigarette lighter). Route the cable neatly along the windshield edge.
3. **Power on.** The device starts automatically when it receives power. A steady green LED indicates that the device has booted and is ready. No buttons to press, no code to run.

Tip: Position the device so the camera has a clear view of your face from roughly arm's length away. Avoid mounting it where the sun visor blocks the lens when flipped down.

Step 2: Download the BLINK App

1. Download **BLINK** from the App Store (iOS) or Google Play Store (Android).
2. Open the app and tap **Create Account**.
3. Enter your email address and create a password.
4. On the role selection screen, tap **Driver**.
5. Enter your **display name** (this is what your fleet operator sees on their dashboard).

Step 3: Join Your Fleet

Your fleet operator will provide you with an **8-character invite code** (for example, `A3B7K9X2`).

1. In the BLINK app, go to **Settings ? Join Fleet**.
2. Enter the invite code and tap **Join**.
3. You are now connected to your fleet. Your operator can see your safety status on their dashboard.

Step 4: Pair with the SleepyDrive Device

1. Make sure Bluetooth is enabled on your phone.
2. Open the BLINK app and go to the **Live Monitor** screen.
3. The app automatically scans for a nearby SleepyDrive device.
4. Connection status will change: **Scanning...** ? **Connecting...** ? **Connected**.
5. Once connected, a green Bluetooth icon appears in the app header.

Troubleshooting: If the app cannot find the device, make sure the device is powered on (green LED), your phone's Bluetooth is on, and you are within ~30 feet of the device.

Step 5: Drive

Once paired, the system is fully automatic:

- **When you are driving normally:** The Live Monitor screen shows your status as **SAFE** with a low fatigue risk percentage.
- **When drowsiness is detected:** You will receive:
- An **audible alarm** from the device's built-in speaker in the truck cab.
- A **phone alert** (vibration + on-screen notification) via Bluetooth.
- A suggestion to **find a nearby rest stop** displayed in the app with directions.
- **If you ignore the warning:** The alarm escalates with increased volume and longer duration. Your fleet operator is also notified in real time.

Understanding Alert Levels

Level	What It Means	What Happens
SAFE	Eyes open, head forward, alert	No action needed
WARNING	Eyes closed for ~1.5 seconds (micro sleep detected)	Short alarm + phone notification
DANGER	Eyes closed for 3+ seconds (sleep event detected)	Loud continuous alarm + phone alert + fleet operator notified

Offline Safety

The SleepyDrive device protects you **even without cell service or internet**:

- The built-in alarm sounds immediately regardless of connectivity.
- If your phone is paired via Bluetooth, you still get phone alerts (Bluetooth works without internet).
- When connectivity returns, any missed events are automatically sent to your fleet operator.

Part 2 - Fleet Operator Setup Guide

Step 1: Create Your Operator Account

1. Download the **BLINK** app or navigate to the BLINK web dashboard.
2. Tap **Create Account** and enter your email and password.
3. On the role selection screen, select **Fleet Operator**.
4. A fleet is automatically created for you with a unique **invite code** (for example, `A3B7K9X2`).

Step 2: Add Drivers to Your Fleet

1. Go to **Fleet Settings** in the app or dashboard.
2. Copy your **invite code** and share it with your drivers (via text, email, or printed card).
3. When a driver enters your code in their BLINK app, they automatically appear on your dashboard.

Note: You can remove a driver from your fleet at any time from the Fleet Settings screen.

Step 3: Install Devices in Your Trucks

For each truck in your fleet:

1. Mount the SleepyDrive device using the visor mount kit.
2. Plug in the 12V power adapter.
3. The device powers on automatically and connects to the SleepyDrive cloud within seconds.
4. Each device has a unique device ID printed on the bottom label. Drivers associate themselves with a device through the app.

Step 4: Monitor Your Fleet

The Fleet Operator Dashboard shows:

Information	Description
Driver List	All drivers in your fleet with their current status
Online / Offline	Whether each device is currently active (heartbeat every 10 seconds)
Fatigue Risk %	Real-time fatigue risk level for each driver
Alert History	Complete log of all drowsiness events with timestamps
Last Seen	When the device last reported its status

Real-Time Alerts

When any driver in your fleet triggers a drowsiness event:

1. Your dashboard highlights the driver in red.
2. An alert notification appears with the driver's name, event type, and timestamp.

3. The alert is logged in the driver's history for later review.

Driver Safety Trends

Use the alert history to identify:

- Drivers who frequently experience fatigue (may need schedule adjustments).
- Times of day when fatigue events are most common.
- Routes that correlate with higher drowsiness rates.

Multi-Truck Operations

Each truck has its own SleepyDrive device with a unique identity. The system scales automatically:

- All devices report to the same cloud backend.
- The dashboard shows all drivers in your fleet simultaneously.
- Each driver's phone pairs with the device in their truck via Bluetooth (1:1 pairing).
- If drivers rotate between trucks, their phone simply re-pairs with the nearest SleepyDrive device.

Part 3 - Frequently Asked Questions

Q: Does the driver need to do anything each time they start a shift? A: No. The device starts automatically when the truck's ignition turns on (or when the 12V port is powered). The phone app auto-reconnects to the device via Bluetooth. The driver just drives.

Q: What happens if my phone battery dies? A: The device's built-in alarm still sounds in the truck cab. The alarm is wired directly to the device and does not depend on the phone.

Q: Does the system record video or store images of the driver? A: No. The AI model processes video frames in real time on the device itself. No video or images are stored, transmitted, or uploaded. Only alert events (text-based: "drowsiness detected at 3:42 PM") are sent to the cloud.

Q: How far away can the Bluetooth connection work? A: Bluetooth Low Energy (BLE) works reliably within approximately 30 feet (10 meters). Inside a truck cab, this is always sufficient.

Q: Can I use the system in my personal vehicle? A: Yes. The device mounts on any visor or mirror and uses a standard 12V power adapter. For personal use, you do not need a fleet operator - the driver app works standalone.

Q: What if the camera cannot see my face (sunglasses, hat, mask)? A: The system will display "Driver not detected" on the app. Standard sunglasses generally do not affect detection. Full face coverings or very dark tinted lenses may reduce accuracy.

Q: Does the device work at night? A: Yes. The device includes an infrared (IR) camera that operates in complete darkness.

Appendix 1 - Problem Formulation

1.1 Conceptualisations

Initial Concept

The core idea is a compact, vehicle-mounted device built around a custom SoC that uses an external camera to continuously monitor the driver's face. When signs of drowsiness are detected - prolonged eye closure, excessive blinking, or head drooping - the system responds through three simultaneous channels: it sends an alert to the driver's smartphone via BLE (triggering an in-app notification and rerouting suggestions), it activates a wired external audible alarm mounted in the cabin to immediately wake the driver, and it publishes the detection event over WiFi via MQTT to a backend system where the fleet operator is notified of the occurrence. The system must operate with minimal latency across all three alert paths.

Key Constraints Identified Early

- **Real-time performance:** Detection-to-alert latency must be low enough that the driver is warned before a dangerous situation develops.
- **Edge inference:** Processing must happen locally on the SoC to avoid dependency on cellular connectivity for detection.
- **Lighting variability:** The system must function at night and in low-light conditions, requiring built-in IR camera support.
- **Non-intrusive:** The system should not distract the driver or require interaction while driving.
- **Cross-platform mobile app:** The alert interface must work on both iOS and Android.
- **Fleet visibility:** Detection events must be forwarded to a backend system so fleet operators can monitor driver fatigue in real time.
- **Redundant alerting:** The system must not rely solely on the phone for driver alerting; a wired external alarm provides a hardware-level backup.
- **Cost:** The total hardware cost should remain feasible for consumer or fleet deployment.

1.2 Brainstorming

The team conducted brainstorming sessions to explore the design space across four major dimensions: hardware platform, ML approach, communication protocol, and mobile app framework. The following captures the ideas generated in each area.

Hardware Platform Options

Custom SoC with WiFi, BLE Raspberry Pi 5 ESP32 Smartphone-only (no external hardware)

ML Model / Framework Options

MediaPipe Face Mesh Custom CNN (eye state classifier) YOLO-Face + landmark regression DeepStream pipeline (NVIDIA)

Communication Protocol Options

MQTT (SoC ? Backend) Direct WebSocket (SoC ? Backend) BLE (SoC ? Phone) Wired GPIO signal (SoC ? External alarm)

Mobile App Framework Options

Flutter React Native Native (Swift + Kotlin)

1.3 Decision Tables

The team used weighted decision matrices to evaluate alternatives across the four major design dimensions. Criteria were weighted based on project priorities (safety-critical real-time performance, development feasibility within two quarters, and cost).

Decision Table 1: Hardware Platform

Criteria (weight)	Custom SoC with WiFi, BLE	RPi 5	ESP32
ML inference speed (0.30)	5	2	1
Power efficiency (0.10)	3	4	5
Cost (0.15)	2	4	5
SDK / tooling (0.20)	5	4	2
Camera/sensor support (0.15)	5	4	2
Community / documentation (0.10)	4	5	4
Weighted Total	4.15	3.40	2.45

Decision: Custom SoC with WiFi, BLE - with GPU-accelerated inference component, which is critical for achieving real-time performance on a DNN-based detection pipeline. The integrated WiFi enables direct MQTT communication with the backend, while BLE provides a low-latency link to the driver's phone.

Decision Table 2: SoC ? Phone Communication Protocol

Criteria (weight)	BLE	Direct WebSocket	MQTT via Phone
Latency (0.25)	**5**	**4**	**3**
Reliability (0.25)	**4**	**3**	**3**
Power efficiency (0.15)	**5**	**2**	**2**
Offline capability (0.15)	**5**	**2**	**2**
Implementation complexity (0.10)	**3**	**4**	**2**
Packet size suitability (0.10)	**4**	**5**	**5**
Weighted Total	**4.40**	**3.15**	**2.55**

Decision: BLE - offers low-latency, low-power, small-packet transfers ideal for sending drowsiness detection events from the SoC to the driver's phone without requiring an internet connection.

Decision Table 3: SoC ? Backend Communication Protocol

Criteria (weight)	MQTT	HTTP POST	Direct WebSocket
Reliability (0.25)	**5**	**4**	**3**
Latency (0.20)	**4**	**3**	**5**
Data persistence (0.20)	**5**	**3**	**2**
Scalability (0.15)	**5**	**3**	**2**
Implementation complexity (0.10)	**3**	**4**	**3**
Offline resilience (0.10)	**5**	**2**	**1**
Weighted Total	**4.55**	**3.25**	**2.80**

Decision: MQTT - provides reliable, lightweight pub/sub messaging with QoS guarantees, well-suited for publishing detection events from the SoC over WiFi to the backend where fleet operators are notified.

Decision Table 4: Mobile App Framework

Criteria (weight)	Flutter	React Native	Native (Swift+Kotlin)
Cross-platform from single codebase (0.30)	**5**	**4**	**1**
Development speed (0.25)	**5**	**4**	**2**
Performance (0.15)	**4**	**3**	**5**
Team familiarity (0.20)	**4**	**3**	**2**
Library ecosystem for BLE (0.10)	**4**	**4**	**5**
Weighted Total	**4.55**	**3.65**	**2.40**

Decision: Flutter - delivers cross-platform support from a single Dart codebase with strong BLE and notification libraries, enabling both driver alerting and rerouting functionality.

Decision Table 5: ML Model / Framework

Criteria (weight)	MediaPipe + EAR	Custom CNN	YOLO-Face
Inference speed (0.30)	4	4	3
Accuracy (0.25)	4	3	3
Integration with SoC (0.20)	4	3	4
Development effort (0.15)	4	1	4
Documentation (0.10)	5	1	4
Weighted Total	4.10	2.80	3.45

Decision: MediaPipe Face Mesh with EAR (Eye Aspect Ratio) and head pose estimation - provides reliable, real-time detection of eye closure, excessive blinking, and head drooping with minimal computational overhead, making it well-suited for edge deployment on the SoC.

1.4 Morphological Chart

The morphological chart below maps each functional requirement of the system to the design alternatives considered, with the **selected option highlighted in bold**.

Function	Option A	Option B	Option C	Option D
Compute platform	**Custom SoC (WiFi + BLE)**	**Raspberry Pi 5**	**RPi + Coral TPU**	**ESP32**
Camera type	**USB webcam (visible light)**	**IR camera (night-capable)**	**Stereo depth camera**	**Smartphone camera**
Face detection	**MediaPipe Face Detector**	**dlib HOG detector**	**YOLO-Face**	**Haar Cascades**
Drowsiness metrics	**EAR only**	**PERCLOS only**	**EAR + head pose estimation**	**Blink frequency only**
Fatigue classification	**Threshold-based (EAR < value)**	**Temporal analysis (EAR + head pose over sliding window)**	**CNN binary classifier**	**Hybrid (threshold + CNN)**
SoC ? phone comm	**BLE**	**Direct WebSocket**	**HTTP POST**	**MQTT via phone**
SoC ? backend comm	**MQTT over WiFi**	**HTTP POST**	**Direct WebSocket**	**None**
SoC ? external alarm	**Wired GPIO signal**	**BLE to alarm module**	**WiFi-controlled relay**	**None**
Backend storage	**PostgreSQL**	**MongoDB**	**SQLite**	**Firebase Realtime DB**
Backend ? fleet operator	**WebSocket gateway**	**FCM push**	**Polling (HTTP)**	**Server-Sent Events**
Mobile framework	**Flutter**	**React Native**	**Native iOS + Android**	**Kotlin Multiplatform**
App alert modality	**Visual popup only**	**Audio alarm + vibration + visual**	**Haptic wearable**	**-**
App rerouting	**In-app rerouting suggestions**	**External maps redirect**	**No rerouting**	**-**
External alarm type	**Audible buzzer/speaker (wired)**	**LED strobe**	**Seat vibration motor**	**-**

Function	Option A	Option B	Option C	Option D
Power source	**Vehicle 12V (via adapter)**	**USB battery bank**	**Hardwired to OBD-II**	**Solar + battery**
Mounting	**Dashboard mount**	**Visor/mirror mount**	**A-pillar clip**	**Rearview mirror replacement**

The selected path through the morphological chart (bold entries) represents the team's final design: a custom SoC with integrated WiFi and BLE, paired with an IR camera, performing EAR-based and head-pose-based drowsiness detection over a sliding temporal window. When drowsiness is detected, the SoC simultaneously (1) sends an alert via BLE to a cross-platform Flutter app that notifies the driver with audio, vibration, and visual alerts and offers rerouting suggestions, (2) activates a wired external audible alarm in the cabin for immediate driver alerting, and (3) publishes the detection event via MQTT over WiFi to a backend that persists data in PostgreSQL and notifies the fleet operator through a WebSocket gateway.

Appendix 2 - Planning

2.1 Basic Plan / Gantt Chart

The project spans two academic quarters at UC Santa Cruz: Winter 2026 (January-March) and Spring 2026 (April-June). The plan is divided into four major phases: Research & Definition, Design & Architecture, Implementation & Prototyping, and Testing & Documentation.

Project Timeline (Text-Based Gantt Chart)

```
Phase / Task                          Jan     Feb     Mar     Apr     May     Jun
W1-W4  W5-W8  W9-W12 W13-16 W17-20 W21-24
-------------------------------------------------------------------------------
---
PHASE 1: Research & Definition
Problem formulation                 ====
Brainstorming & concept gen         ====
Existing product research             =====
Persona development                      ==
Need & goal statements               =====

PHASE 2: Design & Architecture
System architecture design            ======
Communication protocol design             ======
ML pipeline design                    ======
Design document (initial draft)             ====
Mobile app UI mockup                   ==
Hardware procurement                  ==

PHASE 3: Implementation & Prototyping
ML model integration & tuning          ===========
Backend setup (MQTT broker, DB,            ==========
fleet operator WebSocket)
Flutter app development                  =========   =====
External alarm integration                           ======
Hardware mounting / enclosure                            ======
System integration                          ===   =====

PHASE 4: Testing & Documentation
Unit testing (per subsystem)                         =====
Integration testing                                      ======
Design document (final)                                    ======
```

2.2 Division of Labor During Prototyping Phase

The team of six divided responsibilities based on individual expertise and interest areas. The table below shows the primary ownership of each subsystem during the prototyping phase, along with secondary contributors.

Subsystem	Primary Owner(s)	Description
ML Pipeline	Jason, Soham	Face detection model selection, EAR computation, head pose estimation, camera integration, sliding window fatigue classification
SoC ? Phone Communication (BLE)	Jason, Soham	BLE service/characteristic setup on SoC, drowsiness event packet format, connection management
SoC ? Backend Communication (MQTT)	Jason, Soham	MQTT client on SoC, broker setup, event topic/payload design, WiFi connectivity
SoC ? External Alarm (Wired)	Ricardo, Pravin	GPIO-driven alarm trigger, wired buzzer/speaker integration, signal timing
Backend & Fleet Operator Interface	Jason, Soham	MQTT event consumer, PostgreSQL schema design, WebSocket gateway for fleet operator notification
Flutter Mobile App	Pranav, Pravin, Alejandro, Ricardo	UI/UX design, BLE client, alert display (audio/vibration/visual), rerouting suggestions, connection status
Hardware & Enclosure	Alejandro, Pranav	Device mounting, IR camera selection and positioning, power supply (12V vehicle adapter), physical enclosure/housing, external alarm mounting
Documentation & Testing - ML Pipeline & Integration	Jason, Soham	Design document coordination, appendices (problem formulation, planning), test plan development, integration testing across all three alert paths
Documentation & Testing - Mobile App	Pranav, Pravin, Alejandro, Ricardo	Design document coordination, appendices (problem formulation, planning), test plan development

2.3 Collaboration

Repository Structure

The team uses a shared GitHub repository as the single source of truth for all project code and documentation.

Branching Strategy

The team follows a feature-branch workflow:

1. Each team member creates a feature branch from `main` for their work (e.g., `feature/mqtt-broker`, `feature/flutter-alerts`, `feature/ble-service`, `feature/external-alarm`).
2. When a feature is complete, the member opens a Pull Request (PR) for code review.
3. At least one other team member reviews the PR before merging into `main`.
4. Team members regularly pull from `main` into their feature branches to stay in sync and reduce merge conflicts.

Task Management with Teamwork

Beyond the Git repository, the team uses **Teamwork** as the primary project management tool. Teamwork is used for:

- **Task assignment:** Each task is assigned to a team member with a due date. Tasks correspond to the subsystems in the division of labor table above.
- **Milestone tracking:** Key milestones from the Gantt chart are tracked as Teamwork milestones, allowing the team to monitor progress at a glance.
- **Task lists and subtasks:** Larger deliverables (e.g., "Flutter app development") are broken into subtasks (e.g., "Implement BLE client," "Design alert UI," "Add vibration/audio alerts," "Implement rerouting suggestions") and tracked individually.

Communication

- **Regular meetings:** The team meets twice a week to discuss progress, blockers, and upcoming tasks. Meeting notes are shared in a Google Doc
- **Asynchronous communication:** Day-to-day questions and quick updates are handled via Discord
- **Design reviews:** Major design decisions (such as the communication protocol selection documented in Appendix 1) are discussed as a full team before implementation begins.

Tools Summary

Tool	Purpose
GitHub	Version control, code review (PRs), documentation hosting
Teamwork	Task assignment, milestone tracking, project timeline
Discord	Real-time communication
Google Docs	Meeting notes, shared drafts
Figma	Design schematic